Asistente RAG reproducible sobre prensa y Bluesky colombianos con Milvus y LLM locales

Abel Albuez Sánchez, Kelly Joane León Torres, Juan Camilo Torres Peña y Jesús David Romero Melo
Maestría en Inteligencia Artificial, Pontificia Universidad Javeriana
Profesor: Ing. Luis Gabriel Moreno Sandoval, PhD
Repositorio: https://github.com/AbelAlbuez/natural-language-processing-workshops

***Resumen*—Este documento describe el corpus, la segmentación, la arquitectura y el protocolo de evaluación de un asistente de generación aumentada por recuperación (RAG) sobre noticias y publicaciones de Bluesky en español e inglés. El corpus preparado contiene 146.923 documentos y la segmentación por tokens 256/48 produce 577.324 chunks, ambos verificados. La pila usa Milvus standalone v2.6.25 sin MinIO, ya validado, y modelos de lenguaje locales servidos con Ollama. Los pesos del encoder `intfloat/multilingual-e5-small` están descargados y cargan sin conexión. Los embeddings de los 577.324 chunks están calculados y cargados en Milvus con una ingesta por lotes idempotente, y la API responde con citas a los fragmentos recuperados. La configuración de generación quedó congelada con temperatura 0 y `repeat_penalty` 1,1. El gold standard tiene 40 preguntas (34 respondibles sintéticas y 6 no respondibles), divididas en 20 de desarrollo y 20 de prueba y pendientes de revisión humana. Se presentan resultados preliminares en la partición de prueba con una repetición por modelo: Recall@5 de 0,647 a nivel de documento, y modelos que citan fragmentos válidos en entre 5 y 16 de las 17 preguntas respondibles. Con el umbral de 0,80 no hubo abstención en las preguntas fuera del corpus, y la calibración con desarrollo no encontró un umbral que cumpla los criterios fijados. El umbral se mantiene en 0,80 por decisión del autor, con la limitación de que el sistema no se abstiene en las preguntas fuera del corpus. Las referencias bibliográficas están por verificar.**

***Index Terms*—RAG, recuperación de información, corpus bilingüe, segmentación, embeddings, Milvus, LLM locales, evaluación**

I. Problema e hipótesis

El proyecto estudia la recuperación de evidencia en un corpus de prensa y publicaciones sociales colombianas en español e inglés. La hipótesis de trabajo PI6 propone que Recall@5 es menor para Bluesky que para Media Cloud al comparar preguntas del mismo idioma por separado, y que la abstención correcta es menor para preguntas en inglés que para preguntas en español. Es una hipótesis pendiente de evaluación.

El protocolo prevé reportar Recall@k ($k \in \{1, 3, 5, 10\}$), MRR, nDCG@10, abstención correcta y fidelidad de citas, desglosados por fuente esperada e idioma. Las métricas siguen referencias candidatas aún no verificadas [24], [27].

II. Corpus y perfilado (A)

El diagnóstico del proyecto registra 169.248 documentos originales y 146.923 después de la preparación bilingüe: 64.666 en español y 82.257 en inglés. El recuento de 146.923 filas del Parquet preparado se comprobó por metadatos. Las cifras se documentan en `docs/diagnostico_corpus.md` y `docs/prepare_stats_es_en.json`; no se incluye texto de documentos. La Figura 1 desglosa el corpus preparado por fuente e idioma.

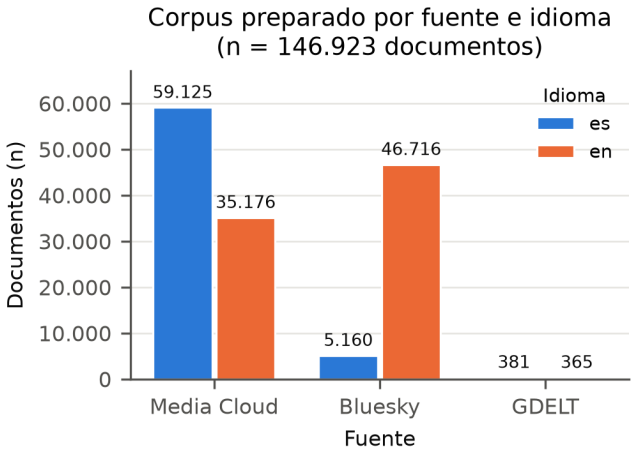


Figura 1. Documentos del corpus preparado (146.923) por fuente e idioma. Las barras de GDELT (381 en español y 365 en inglés) son casi invisibles a esta escala.

Cuadro I
Tamaño del corpus preparado.

Etapa o idioma	Documentos
Originales	169.248
Preparados, total	146.923
Español	64.666
Inglés	82.257

III. Segmentación (B)

La segmentación por tokens usa ventanas de 256 tokens con solapamiento de 48 y produce 577.324 chunks en `data/chunks_tok256.parquet`. Al retokenizar, 40.474 chunks superan 256 tokens, con un máximo de 258. Por documento, la mediana es de 3 chunks, el percentil 95 de 10 y el máximo de 426. Estas cifras están verificadas. La Figura 2 muestra la distribución de chunks por documento y la Figura 3, la de tokens por chunk al retokenizar.

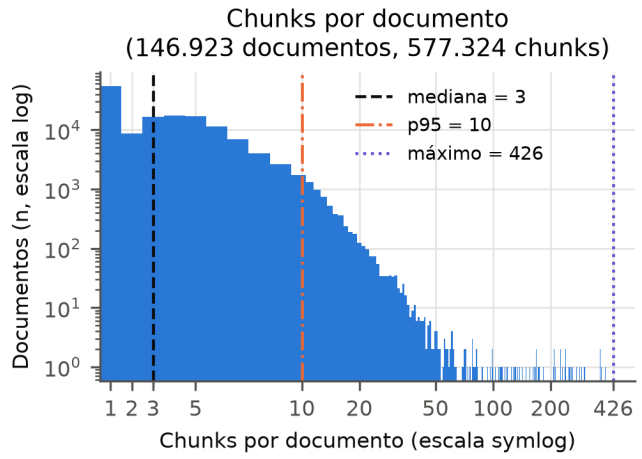


Figura 2. Histograma de chunks por documento (146.923 documentos, 577.324 chunks), con eje vertical logarítmico. Las líneas marcan la mediana (3), el percentil 95 (10) y el máximo (426).

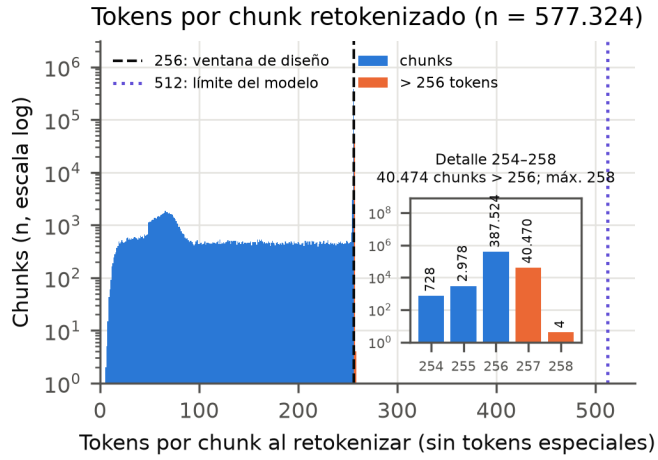


Figura 3. Tokens por chunk al retokenizar los 577.324 chunks sin tokens especiales. Las líneas marcan la ventana de diseño (256) y el límite del modelo (512). El recuadro amplía la cola: 40.474 chunks superan 256 tokens y el máximo es 258.

IV. Embeddings (C)

El proyecto usa `intfloat/multilingual-e5-small` con el prefijo `passage:` para documentos y `query:` para consultas [6], [7]. Los pesos se descargaron en la caché local de Hugging Face, fijados a la revisión `614241f`. Se bajaron solo los archivos necesarios para PyTorch y sentence-transformers: unos 493 MB, o 470 MiB en disco. La carga sin conexión con `AutoTokenizer` y `AutoModel` (`local_files_only=True`) funcionó: el modelo tiene una dimensión oculta de 384 y 117.653.760 parámetros.

Los embeddings del piloto ya están calculados: 500 documentos y 1.904 chunks, con el prefijo `passage:`, lotes de 32 y normalización L2, en CPU. El cálculo tardó 22,4 s (11,8 ms por chunk) con un pico de memoria de 1,9 GB. Con la misma tasa, los 577.324 chunks del corpus completo tomarían unas 1,9 h. Son mediciones operativas de un solo equipo, no resultados de recuperación. Después se calcularon los embeddings del corpus completo: 577.324 vectores de dimensión 384, procesados en 29 bloques reanudables de 20.000 chunks. Se verificó que los `chunk_id` son únicos y coinciden con los de la segmentación, que no hay valores NaN y que todas las normas están a menos de 10^{-6} de 1. La comparación con otros encoders sigue pendiente.

V. Milvus (D)

La pila usa Milvus standalone v2.6.25 con etcd embebido y almacenamiento local, sin MinIO. Esta configuración está validada. La colección `rag_e5s_tok256_v1` contiene los 577.324 chunks del corpus completo (Sección VI).

VI. Implementación y configuración congelada

Ingesta por lotes. La primera versión de la ingesta leía todos los vectores a la vez y el proceso llegaba a 9,07 GB de memoria. La versión actual lee el Parquet de embeddings en lotes de 20.000 filas, toma de la segmentación solo los metadatos de esos chunks y hace *upsert* por `chunk_id`. Con ella, la carga completa tardó 115 s y la memoria máxima del proceso bajó a 2,41 GB. Una segunda carga idéntica dejó la colección en las mismas 577.324 filas, contadas con `count(*)`, lo que confirma que la ingesta es idempotente.

Memoria de Milvus. Con el límite inicial de 4 GiB, Milvus se detuvo por falta de memoria al recibir la segunda carga completa, porque el *upsert* conserva las versiones reemplazadas hasta la compactación. Con el límite en 8 GiB las dos cargas terminaron sin reinicios.

API. FastAPI expone `/health`, `/retrieve` y `/ask`. El conteo de filas de `/health` usa `count(*)`, porque las estadísticas de Milvus incluyen las filas reemplazadas hasta la compactación. Cuando Ollama corta una generación por repetición, `/ask` responde con un error específico (HTTP 502) en lugar de uno genérico.

Configuración congelada. Los valores fijos para todas las condiciones están en la sección 9 de `docs/protocolo_evaluacion.md`: `prompt_v1` con su huella sha256, $k = 5$, chunks de 256/48, el encoder E5 en una revisión fija, HNSW con IP, la partición 20/20 y, para la generación, temperatura 0 y `repeat_penalty` 1,1 en los cuatro modelos. La penalización se añadió porque, con temperatura 0 y sin ella, `gemma2:2b` entró en un bucle de repetición con un prompt real y Ollama cortó la generación ("token repeat limit reached"). Con 1,1, `gemma2:2b` responde con citas que apuntan a fragmentos recuperados. El umbral de abstención se mantiene en 0,80 por decisión del autor, porque la calibración con desarrollo no encontró un valor que cumpla los criterios (Sección IX).

Hallazgos de la prueba técnica. En la prueba de `repeat_penalty`, con un solo prompt real y las dos configuraciones, `qwen2.5:1.5b` no citó ningún fragmento (0 citas; en el experimento, con las 17 respondibles de prueba, citó en 5) y `llama3.2:1b` generó respuestas muy cortas, de

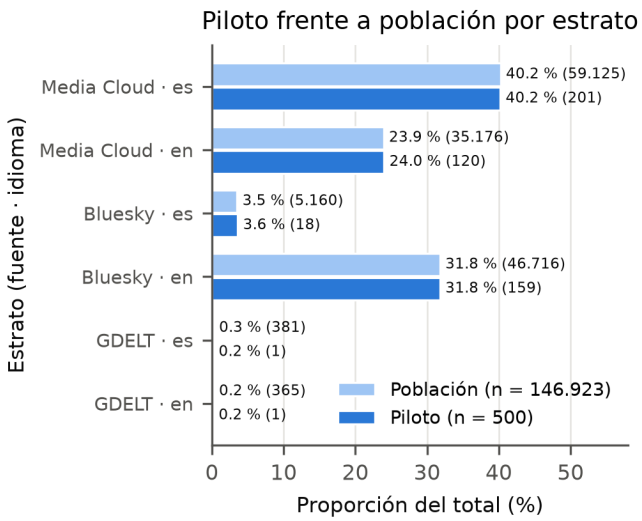


Figura 4. Proporción de cada estrato fuente × idioma en el piloto (500 documentos: 220 en español y 280 en inglés) y en la población (146.923 documentos). Entre paréntesis, los conteos.

7 a 9 tokens. Son observaciones de una comprobación técnica, no resultados experimentales, pero anticipan diferencias que la evaluación deberá medir.

Estado de la evaluación. El gold standard sintético y su constructor, el validador, las funciones de métricas, el script del experimento, la calibración del umbral con desarrollo y el análisis de resultados están en el repositorio y probados. El experimento con los cuatro modelos sobre la partición de prueba se presenta en la Sección IX. Siguen pendientes la revisión humana del gold standard, un segundo anotador, la calibración del umbral con anotaciones humanas (0,80 se mantiene por decisión del autor), la evaluación de la corrección de las respuestas y las repeticiones.

VII. Diseño experimental (5)

El plan propuesto empieza con un piloto de 500 documentos y escala a 10.000 y luego al corpus completo solo si el hardware lo permite. El piloto se seleccionó con semilla 42, estratificado por fuente (`source`) e idioma (`lang`). La Tabla II solo informa conteos; la Figura 4 compara sus proporciones con las de la población.

Cuadro II
Conteos del piloto de 500 documentos por fuente e idioma.

Fuente	es	en	Total
Bluesky	18	159	177
GDELT	1	1	2
Media Cloud	201	120	321
Total	220	280	500

Entre condiciones experimentales solo cambia el LLM; corpus, segmentación, encoder, índice, `top-k` y prompt se mantienen fijos. Según `docs/EVALUACION.md`, cada condición se repetirá al menos tres veces, registrando el arranque en frío y en caliente. Los modelos locales previstos se servirán con Ollama [28]–[30]. Los tiempos de cálculo de embeddings e ingesta están en la Sección VI, y las métricas de la partición de prueba, con una sola repetición, en la Sección IX. Las repeticiones y el registro de arranque en frío y en caliente siguen pendientes. Las referencias de evaluación automática son candidatas por verificar [5], [24], [25].

VIII. Gold standard y partición (6)

El gold standard vigente (`docs/gold_standard.csv`) contiene 40 preguntas: 34 respondibles sintéticas y 6 no respondibles. Cada respondible se basa en un único chunk de la segmentación, y su respuesta de referencia se deduce solo de ese chunk. El documento de ese chunk es la evidencia. Las no respondibles tratan temas ajenos al corpus (recetas, deportes, programación) y no tienen fuente, tema, respuesta ni evidencia. El tema (una de las 18 etiquetas de `collection_keyword_label`) solo se asigna en Bluesky, porque Media Cloud y GDELT no tienen esa etiqueta. Las 40 preguntas están **pendientes de revisión humana**. El validador (`scripts/validar_gold_standard.py`) aprueba las 13 verificaciones. La Tabla III resume la distribución.

Cuadro III
Distribución del gold standard vigente.

Tipo y fuente	es	en	Total
Media Cloud	13	8	21
Bluesky	4	8	12
GDELT	1	0	1
No respondibles	3	3	6
Total	21	19	40

La partición, registrada en `docs/protocolo_evaluacion.md`, divide las 40 preguntas en 20 de desarrollo y 20 de prueba con las listas de IDs fijadas con semilla 42. Las cuotas por fuente e idioma se aplican solo a las respondibles, y las no respondibles se reparten 3 y 3 (Tabla IV). El único caso GDELT en español va a prueba, porque un estrato de un solo caso no puede quedar en ambas particiones. Los parámetros se ajustan solo con desarrollo y se congelan antes de evaluar con prueba.

Cuadro IV
Partición desarrollo/prueba del gold standard vigente.

Fuente esperada	Idioma	Desarrollo	Prueba	Total
Bluesky	en	4	4	8
Bluesky	es	2	2	4
GDELT	es	0	1	1
Media Cloud	en	4	4	8
Media Cloud	es	7	6	13
No respondibles		3	3	6
Total		20	20	40

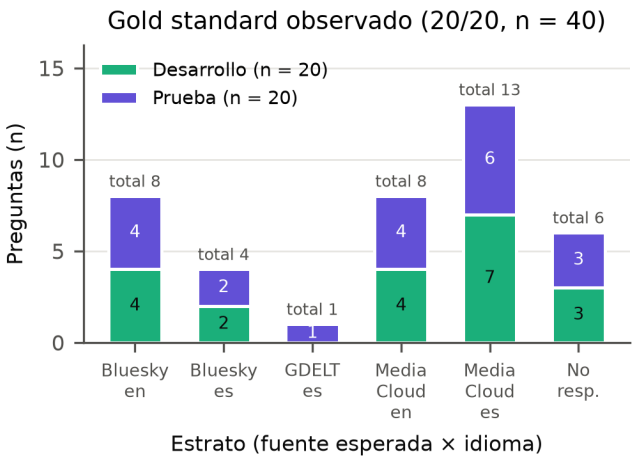


Figura 5. Distribución observada del gold standard vigente: 34 respondibles por fuente esperada e idioma y 6 no respondibles, con 20 preguntas de desarrollo y 20 de prueba. Coincide con la Tabla IV.

La evaluación humana independiente, la rúbrica y la resolución de desacuerdos están pendientes.

IX. Resultados preliminares (partición de prueba)

Se evaluaron las 20 preguntas de prueba con los cuatro modelos, una repetición por modelo (80 corridas), con la configuración congelada: temperatura 0, `repeat_penalty` 1,1, $k = 5$ y umbral de abstención 0,80, el mismo para todos los modelos. Todas las corridas registradas usaron esos valores. Las métricas se calcularon con `scripts/analisis_resultados.py`, que usa las funciones de `scripts/metricas_rag.py`. Una corrida con error cuenta como fallo y permanece en el denominador.

Recuperación. La recuperación no depende del modelo. Se mide a nivel de documento sobre las 17 respondibles, con el documento de evidencia como único relevante (Tabla V). Como el sistema recupera $k = 5$ fragmentos, se reportan las métricas en 5; las variantes en 1, 3 y 10 y nDCG@10 previstas en la Sección I quedan pendientes.

Cuadro V
Recuperación en la partición de prueba (top-5, nivel de documento).

Grupo	n	Recall@5	MRR	nDCG@5
Todas	17	0,647	0,578	0,596
Media Cloud	10	0,700	0,583	0,613
Bluesky	6	0,500	0,500	0,500
GDELT	1	1,000	1,000	1,000
Inglés	8	0,750	0,667	0,688
Español	9	0,556	0,500	0,515

Generación. La Tabla VI resume las 20 corridas de cada modelo. "Cita válida" significa que la cita $[n]$ apunta a un fragmento entregado al modelo; las 85 citas válidas apuntan a `chunk_id` recuperados para su pregunta, y no hubo citas fuera de rango. "Cita a la evidencia" significa que al menos una cita apunta al documento de evidencia. La corrección de las respuestas frente a la referencia no se evaluó.

Cuadro VI
Generación por modelo en la partición de prueba (17 respondibles y 3 no respondibles; una repetición). Err.: corridas con error; Cita: respondibles con al menos una cita válida; Evid.: respondibles que citan el documento de evidencia; Inv.: citas inválidas sobre el total; NR: abstenciones correctas en no respondibles; t: tiempo medio en segundos.

Modelo	Err.	Cita	Evid.	Inv.	NR	t
gemma2:2b	1	14/17	10/17	0/24	0/3	23,0
qwen2.5:1.5b	0	5/17	4/17	0/6	0/3	24,5
llama3.2:1b	0	16/17	9/17	0/46	0/3	16,7
llama3.2:3b	0	6/17	5/17	0/9	0/3	39,5

Ningún modelo se abstuvo en las respondibles, y ninguno se abstuvo en las 3 no respondibles: con el umbral 0,80 todas superaron el umbral y llegaron al LLM. Las medianas de tiempo fueron 23,8; 23,2; 13,0 y 37,8 s, en el orden de la tabla.

Tiempos de generación. Hay dos mediciones distintas, y no deben confundirse. (1) En el experimento de 80 corridas (Tabla VI), el tiempo medio por respuesta fue de 16,7 a 39,5 s según el modelo, con un máximo de 56,8 s (`llama3.2:3b`); sale del registro del experimento. (2) En una prueba manual de la interfaz con `gemma2:2b`, una sola consulta (una pregunta sobre deforestación, en CPU) tardó 69,5 s de generación. La fuente es la captura de la interfaz (generación de 69,46 s, total de 69,8 s) y los logs de Ollama de la misma consulta (tiempo total de 66,8 s: 21,4 s de evaluación del prompt y 45,4 s de generación). Es una observación aislada, no una cifra general del sistema.

Error de gemma2:2b en G34. La corrida de G34 con `gemma2:2b` devolvió HTTP 503 en 0,51 s. El registro de Ollama muestra que el proceso `llama-server` recibió `SIGKILL` y la generación terminó con `EOF`. La corrida no se reintentó: `gemma2:2b` tuvo 19 corridas exitosas de 20, y el fallo cuenta en el denominador de sus métricas, que se calculan sobre las 20.

Calibración del umbral (solo desarrollo). Con los scores máximos de recuperación de las 20 preguntas de desarrollo, **ningún umbral cumple a la vez** abstenerse en las 3 no respondibles y responder al menos el 90 % de las respondibles. Con $t \geq 0{,}876$ se abstiene en las 3 no respondibles, pero responde solo 13 de 17 respondibles (76,5 %). Con $t \leq 0{,}8465$ responde 16 de 17, pero se abstiene solo en 1 de 3. Con $t = 0{,}80$ responde las 17 y no se abstiene en ninguna. **Decisión del autor:** el umbral del sistema se mantiene en 0,80. No se ajustó ningún parámetro con la partición de prueba.

Análisis complementario (no usado para decidir). Con los scores de la partición de prueba, $t = 0{,}80$ responde 17 de 17 respondibles y abstiene 0 de 3 no respondibles; $t = 0{,}8465$, 15 de 17 y 3 de 3; y $t = 0{,}876$, 9 de 17 y 3 de 3. Se reporta

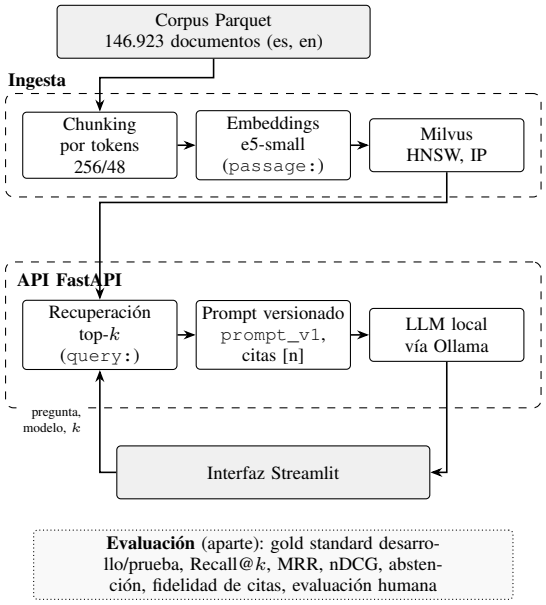


Figura 6. Arquitectura implementada. La interfaz Streamlit se verificó en ejecución con `gemma2:2b`: el servicio respondió a su chequeo de salud y sirvió la página, alcanzó la API dentro de Docker, y `/ask` con una pregunta de desarrollo respondió en 21,2 s con una cita válida a un fragmento recuperado. La ingesta segmenta el corpus, calcula embeddings E5 y los indexa en Milvus; la API recupera fragmentos, arma el prompt versionado y consulta el LLM local; la interfaz solo habla con la API. La evaluación es un bloque aparte que consume las respuestas registradas.

solo como información: elegir el umbral con estos números contaminaría la evaluación de prueba.

X. Decisiones de arquitectura

La configuración implementada combina los documentos preparados, la segmentación 256/48, embeddings E5 con prefijo de pasaje, Milvus como almacén vectorial, una API FastAPI, una interfaz Streamlit (verificada en ejecución con `gemma2:2b`) y LLM locales servidos con Ollama. La Figura 6 muestra el flujo, con la evaluación como bloque aparte.

Encoder. Se usa `multilingual-e5-small`, un encoder multilingüe entrenado con los prefijos `query:` y `passage:` para distinguir consultas de pasajes [7]; el código aplica cada prefijo en la consulta y en la indexación. BGE-M3 es una alternativa multilingüe que además combina varias funciones de recuperación [8]. La comparación controlada entre ambos sigue pendiente, por lo que la elección del encoder todavía no se considera justificada empíricamente.

Índice. Los vectores se normalizan con L2, de modo que el producto interno (IP) equivale al coseno. La colección usa un índice HNSW con métrica IP ($M = 16$, `efConstruction` $= 200$ y `ef` $= 64$ en búsqueda). HNSW organiza los vectores en grafos jerárquicos de mundo pequeño navegables y permite una búsqueda aproximada de vecinos más cercanos eficiente y robusta [10], lo que evita comparar cada consulta con los 577.324 vectores del corpus completo.

Abstención. Un sistema de preguntas y respuestas debe reconocer cuándo la evidencia no permite responder [22], y la abstención es un comportamiento que se estudia de forma específica en los LLM [23]. Self-RAG propone que el modelo decida cuándo recuperar y critique lo recuperado [21]. Además, los modelos de lenguaje aumentados por recuperación deben ser robustos ante contexto irrelevante [19], lo que motiva no entregar al LLM fragmentos con baja puntuación. Aquí se usa una regla más simple: si la mejor puntuación de recuperación queda por debajo de un umbral, el sistema responde que no hay evidencia suficiente y no llama al LLM. El umbral de 0,80 es provisional: las puntuaciones de E5 no se han calibrado en este corpus para decidir la abstención, y el valor se fijará con las 20 preguntas de desarrollo antes de evaluar con prueba.

Contexto. Los LLM aprovechan peor la información situada en el medio de contextos largos [18]. Por eso los fragmentos entran ordenados por puntuación y completos, hasta un presupuesto configurable de tokens (2.048 por defecto); nunca se corta un fragmento. La plantilla `prompt_v1` numera cada fragmento como [n] y pide citarlo, y la API comprueba que cada cita corresponda a un fragmento entregado.

Evaluación de fidelidad. La fidelidad de las respuestas y de las citas se evaluará con anotación humana. RAGAS [24] y ARES [25] proponen marcos automáticos para evaluar sistemas RAG, y el uso de un LLM como juez [26] servirá solo como complemento, contrastado con la evaluación humana.

Inyección de prompt. Los documentos recuperados pueden contener instrucciones que manipulen al LLM (inyección indirecta de prompt) [35], y un atacante podría insertar textos diseñados para alterar las respuestas de un sistema RAG [36]. El prompt ordena ignorar las instrucciones contenidas en los fragmentos; es una mitigación parcial y no se ha evaluado.

Las referencias sobre recuperación híbrida y embeddings son candidatas por verificar [1]–[4], [8], [9].

XI. Conclusiones preliminares

Con el corpus completo indexado, la recuperación encuentra el documento de evidencia entre los cinco primeros en 11 de 17 preguntas de prueba (Recall@5 0,647), En el agregado, lo hace mejor en Media Cloud (0,700) que en Bluesky (0,500). PI6, sin embargo, compara fuentes dentro de cada idioma: en inglés, Bluesky obtiene 0,500 y Media Cloud 1,000 ($n = 4$ y 4); en español, ambas obtienen 0,500 ($n = 2$ y 6). Solo la comparación en inglés va en la dirección de PI6, y con estos tamaños y una sola repetición no permite concluir nada. La parte de PI6 sobre abstención no se puede evaluar, porque no hubo abstenciones. Entre los modelos, `llama3.2:1b` y `gemma2:2b` citan fragmentos válidos en la mayoría de las respondibles, mientras que `qwen2.5:1.5b` y `llama3.2:3b` citan en menos de la mitad. Ningún modelo produjo citas fuera de rango. La abstención por umbral sobre el score máximo de E5 no separa las preguntas fuera del corpus de las respondibles: con 0,80 ninguna no respondible se abstuvo, y en desarrollo ningún umbral cumple las dos condiciones fijadas. La abstención necesita otra señal o una calibración con preguntas anotadas por personas.

XII. Limitaciones y amenazas a la validez

Las preguntas del gold standard son sintéticas, redactadas con asistencia de IA a partir de un único chunk cada una, y no tienen validación humana. Hay una sola repetición por modelo, sin intervalos de confianza. El tema solo existe en Bluesky, así que no se puede desglosar por tema en Media Cloud ni en GDELT. La generación en CPU es lenta: en el experimento de 80 corridas, el tiempo medio por respuesta fue de 16,7 a 39,5 s según el modelo, con un máximo de 56,8 s (`llama3.2:3b`); en una prueba aislada de la interfaz con `gemma2:2b`, una consulta tardó 69,5 s de generación. `qwen2.5:1.5b` cita poco: con un solo prompt de depuración no citó ningún fragmento, y en el experimento solo 5 de sus 17 respuestas respondibles incluyen una cita válida. Los estratos son pequeños: 6 preguntas de Bluesky, 1 de GDELT y 3 no respondibles en prueba. El umbral de abstención de 0,80 no está calibrado con anotaciones humanas, y la calibración con desarrollo no encontró ningún valor que cumpla a la vez los dos criterios (abstenerse en todas las no respondibles y responder al menos el 90 % de las respondibles). Con el umbral que se mantiene, 0,80, **el sistema no se abstiene en ninguna de las preguntas fuera del corpus**: las 3 de prueba llegan al LLM con los cuatro modelos. La "cita válida" solo comprueba que la cita apunta a un fragmento entregado, no que respalde la afirmación. La corrección de las respuestas no se evaluó. La preparación, los filtros de idioma y la segmentación determinan la población evaluada; cualquier cambio exige documentar una nueva versión de los datos. Todas las referencias permanecen por verificar.

XIII. Pendientes

- Revisión humana de las 40 preguntas del gold standard.
- Un segundo anotador, con medición del acuerdo y resolución de desacuerdos.
- Verificación de las 36 referencias contra su fuente original.
- Calibración del umbral de abstención con anotaciones humanas; mientras tanto se mantiene 0,80 por decisión del autor.
- Evaluación de la corrección de las respuestas frente a la referencia.
- Repeticiones por condición, con intervalos de confianza.
- Experimentos a mayor escala.
- Justificación empírica del encoder, con la comparación controlada frente a otros encoders.

Referencias

[1] S. Robertson y H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," 2009, doi:10.1561/1500000019. *Por verificar.*

[2] G. V. Cormack, C. L. A. Clarke y S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," 2009, doi:10.1145/1571941.1572114. *Por verificar.*

[3] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," 2020, arXiv:2004.04906. *Por verificar.*

[4] T. Formal, B. Piwowarski y S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," 2021, arXiv:2107.05720. *Por verificar.*

[5] N. Thakur et al., "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," 2021, arXiv:2104.08663. *Por verificar.*

[6] L. Wang et al., "Text Embeddings by Weakly-Supervised Contrastive Pre-training," 2022, arXiv:2212.03533. *Por verificar.*

[7] L. Wang et al., "Multilingual E5 Text Embeddings: A Technical Report," 2024, arXiv:2402.05672. *Por verificar.*

[8] J. Chen et al., "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," 2024, arXiv:2402.03216. *Por verificar.*

[9] N. Muennighoff et al., "MTEB: Massive Text Embedding Benchmark," 2022, arXiv:2210.07316. *Por verificar.*

[10] Y. Malkov y D. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE TPAMI*, 2018. *Por verificar.*

[11] X. Zhang et al., "MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages," 2023, arXiv:2210.09984. *Por verificar.*

[12] R. Nogueira y K. Cho, "Passage Re-ranking with BERT," 2019, arXiv:1901.04085. *Por verificar.*

[13] O. Khattab y M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," 2020, arXiv:2004.12832. *Por verificar.*

[14] W. Sun et al., "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents," 2023, arXiv:2304.09542. *Por verificar.*

[15] Anthropic, "Introducing Contextual Retrieval," 2024, https://www.anthropic.com/news/contextual-retrieval. *Por verificar.*

[16] M. Günther et al., "Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models," 2024, arXiv:2409.04701. *Por verificar.*

[17] P. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," 2024, arXiv:2401.18059. *Por verificar.*

[18] N. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," *TACL*, 2024. *Por verificar.*

[19] O. Yoran et al., "Making Retrieval-Augmented Language Models Robust to Irrelevant Context," 2023, arXiv:2310.01558. *Por verificar.*

[20] S.-Q. Yan et al., "Corrective Retrieval Augmented Generation (CRAG)," 2024, arXiv:2401.15884. *Por verificar.*

[21] A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," 2023, arXiv:2310.11511. *Por verificar.*

[22] P. Rajpurkar, R. Jia y P. Liang, "Know What You Don't Know: Unanswerable Questions for SQuAD," 2018, arXiv:1806.03822. *Por verificar.*

[23] B. Wen et al., "Know Your Limits: A Survey of Abstention in Large Language Models," 2024, arXiv:2407.18418. *Por verificar.*

[24] S. Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation," 2023, arXiv:2309.15217. *Por verificar.*

[25] J. Saad-Falcon et al., "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," 2023, arXiv:2311.09476. *Por verificar.*

[26] L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023, arXiv:2306.05685. *Por verificar.*

[27] K. Järvelin y J. Kekäläinen, "Cumulated Gain-Based Evaluation of IR Techniques," 2002, doi:10.1145/582415.582418. *Por verificar.*

[28] Qwen Team, "Qwen2.5 Technical Report," 2024, arXiv:2412.15115. *Por verificar.*

[29] Gemma Team, "Gemma 2: Improving Open Language Models at a Practical Size," 2024, arXiv:2408.00118. *Por verificar.*

[30] A. Grattafiori et al. (Llama Team), "The Llama 3 Herd of Models," 2024, arXiv:2407.21783. *Por verificar.*

[31] M. Abdin et al., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," 2024, arXiv:2404.14219. *Por verificar.*

[32] A. Z. Broder, "On the Resemblance and Containment of Documents," 1997, doi:10.1109/SEQUEN.1997.666900. *Por verificar.*

[33] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," 2002, doi:10.1145/509907.509965. *Por verificar.*

[34] J. Leskovec, L. Backstrom y J. Kleinberg, "Meme-tracking and the Dynamics of the News Cycle," 2009, doi:10.1145/1557019.1557077. *Por verificar.*

[35] K. Greshake et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," 2023, arXiv:2302.12173. *Por verificar.*

[36] W. Zou et al., “PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models,” 2024, arXiv:2402.07867. *Por verificar*.